Express.js — Zero to Master

22 topics, grouped as Beginner → Intermediate → Advanced → Master. Work through the topics in order; type every code example yourself and finish the practice tasks.

Contents

Beginner (6 topics)

01 Introduction and Setup

02 Your First Server and Routes

03 Request and Response Objects

04 Middleware

05 Routers and Project Structure

06 Serving Static Files and Templates

Intermediate (6 topics)

07 REST API Design

08 Validation

09 Error Handling Middleware

10 Connecting to a Database

11 Environment Configuration

12 CORS and Cookies

Advanced (6 topics)

13 Authentication and Authorization

14 File Uploads

15 Security Hardening

16 Logging and Monitoring

17 Testing APIs

18 API Documentation

Master (4 topics)

19	Layered Architecture
20	Caching, Queues and Performance
21	Deployment with Docker and CI/CD
22	Real Project: Task/Blog/E-commerce API

Beginner Level

01. Introduction and Setup

DEFINITION

Express is a minimal web framework for Node.js for building APIs and web apps.

SYNTAX

```
npm i express
```

EXAMPLE

```
npm i express
import express from 'express';
const app = express();
app.listen(3000);
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A small reception desk for a shop: it receives every visitor request and directs it to the right person.

தமிழ்

கடைக்கான சிறிய வரவேற்பு மேசை: ஒவ்வொரு பார்வையாளர் கோரிக்கையையும் பெற்று சரியான நபருக்கு அனுப்புகிறது.

PROS

- Minimal and flexible
- Huge middleware ecosystem
- Easy to learn

CONS

- No built-in structure
- Security and validation are your job

ADVANCED / NEW FEATURES

- Express 5 is the default release (async errors handled, Node 18+)

02. Your First Server and Routes

DEFINITION

Define routes with `app.get/post/put/delete(path, handler)`.

SYNTAX

```
app.get('/', (req, res) => res.send('Hi'))
```

EXAMPLE

```
app.get('/', (req, res) => res.send('Hello Express'));
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A shop's first counter: when someone asks 'hello', it answers 'Hi'.

தமிழ்

கடையின் முதல் கவுண்டர்: யாராவது 'வணக்கம்' என்றால் 'வணக்கம்' என்று பதிலளிக்கும்.

PROS

- Very little code
- Clear routing

CONS

- Large apps need structure

ADVANCED / NEW FEATURES

- Express 5 path syntax changes (path-to-regexp v8)

03. Request and Response Objects

DEFINITION

`req.params`, `req.query`, `req.body`, `req.headers`; `res.json`, `res.status`, `res.redirect`.

SYNTAX

```
req.params / req.query / req.body
res.status(201).json(obj)
```

EXAMPLE

```
app.get('/users/:id', (req, res) => res.json({ id: req.params.id, q: req.query.q }));
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A letter has a to-address (params), subject (query) and message (body); the reply is the response.

தமிழ்

கடிதத்தில் முகவரி (params), பொருள் (query), செய்தி (body) இருக்கும்; பதில் response.

PROS

- Simple API
- JSON helpers

CONS

- Must validate input yourself

ADVANCED / NEW FEATURES

- `req.query` is a getter in Express 5
- `res.json` improvements

04. Middleware

DEFINITION

Functions `(req, res, next)` run in order. Built-ins: `express.json()`, `express.static()`.

SYNTAX

```
app.use((req, res, next) => { next(); })
```

EXAMPLE

```
app.use(express.json());
app.use((req, res, next) => { console.log(req.method, req.url); next(); });
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A chain of checkpoints on a road: toll, police, weighing station; each middleware does one job and passes on.

தமிழ்

சாலையில் சுங்கம், காவல், எடை நிலையம் எனத் தொடர் சோதனைச் சாவடிகள் போல middleware ஒவ்வொன்றும் ஒரு வேலை செய்து அடுத்ததுக்கு அனுப்பும்.

PROS

- Composable pipeline
- Reusable logic

CONS

- Order matters
- Forgetting `next()` hangs requests

ADVANCED / NEW FEATURES

- Async middleware errors auto-forwarded in v5

05. Routers and Project Structure

DEFINITION

Group routes with `express.Router()` and mount them under prefixes.

SYNTAX

```
const r = express.Router(); app.use('/users', r);
```

EXAMPLE

```
const router = express.Router();
router.get('/', listUsers);
app.use('/api/users', router);
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Separate counters for different departments in a government office; routers group related routes.

தமிழ்

அரசு அலுவலகத்தில் துறைகளுக்கு தனி கவுண்டர்கள் போல routers தொடர்புடைய routes-ஐ தொகுக்கின்றன.

PROS

- Organised code
- Mountable modules

CONS

- Too many routers add indirection

ADVANCED / NEW FEATURES

- Modular routers with ESM imports

06. Serving Static Files and Templates

DEFINITION

Serve assets with `express.static` and render views with EJS/Pug.

SYNTAX

```
app.use(express.static('public'))
```

EXAMPLE

```
app.use(express.static('public'));
app.set('view engine', 'ejs');
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A notice board outside the shop with fixed posters; static files are served without extra code.

தமிழ்

கடைக்கு வெளியே நிலையான சுவரொட்டிகள் உள்ள பலகை போல static கோப்புகள் கூடுதல் குறியீடு இல்லாமல் வழங்கப்படும்.

PROS

- Easy asset serving
- Caching headers

CONS

- Better handled by CDN/Nginx in production

ADVANCED / NEW FEATURES

- Static `dotfiles` safer defaults

Intermediate Level

07. REST API Design

DEFINITION

Use nouns for resources, correct verbs and status codes (200, 201, 400, 401, 404, 500), pagination and versioning.

SYNTAX

```
POST /items -> 201
GET /items/1 -> 200
```

EXAMPLE

```
app.post('/api/todos', (req, res) => res.status(201).json(todo));
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A menu where each dish has a clear name and price: REST API resources use clear URLs and standard HTTP verbs.

தமிழ்

ஒவ்வொரு உணவுக்கும் தெளிவான பெயர், விலை உள்ள மெனு போல REST API வளங்கள் தெளிவான URL, HTTP verbs பயன்படுத்தும்.

PROS

- Standard and predictable
- Easy to consume

CONS

- Over/under-fetching
- Versioning effort

ADVANCED / NEW FEATURES

- OpenAPI 3.1 tooling
- JSON:API conventions

08. Validation

DEFINITION

Never trust input; validate bodies with Zod, Joi or express-validator.

SYNTAX

```
schema.parse(req.body)
```

EXAMPLE

```
const schema = z.object({ title: z.string().min(1) });
const data = schema.parse(req.body);
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A bank clerk checks every field before accepting the form; validation protects your server.

தமிழ்

வங்கி எழுத்தர் படிவத்தின் ஒவ்வொரு புலத்தையும் சரிபார்த்து ஏற்பது போல validation உங்கள் server-ஐப் பாதுகாக்கிறது.

PROS

- Prevents bad data
- Clear error messages

CONS

- Extra code

ADVANCED / NEW FEATURES

- Zod/Valibot typed schemas
- OpenAPI-driven validation

09. Error Handling Middleware

DEFINITION

Four-argument middleware `(err, req, res, next)` centralises errors; wrap async handlers.

SYNTAX

```
app.use((err, req, res, next) => { })
```

EXAMPLE

```
app.use((err, req, res, next) => res.status(err.status ?? 500).json({ error: err.message }));
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A complaint desk at the end of the process: any error from any counter ends up here with a polite response.

தமிழ்

செயல்முறையின் இறுதியில் உள்ள புகார் மேசை: எந்த கவுண்டரில் பிழை வந்தாலும் இங்கு மரியாதையான பதிலுடன் முடியும்.

PROS

- Central error format
- Cleaner routes

CONS

- Must be registered last
- Easy to leak stack traces

ADVANCED / NEW FEATURES

- Async errors forwarded automatically in Express 5

10. Connecting to a Database

DEFINITION

Use Mongoose (MongoDB), `pg` / `mysql2` or Prisma; keep data access in repository/service layers.

SYNTAX

```
await Model.find({})
```

EXAMPLE

```
const user = await prisma.user.findUnique({ where: { id: Number(req.params.id) } });
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

The shop owner's ledger is kept in the back office; Express routes call the database through a service layer.

தமிழ்

கடை உரிமையாளரின் பதிவேடு பின்புற அலுவலகத்தில் இருக்கும்; Express routes service layer வழியாக database-ஐ அழைக்கும்.

PROS

- Many ORMs/drivers
- Separation of concerns

CONS

- Queries in routes become messy

ADVANCED / NEW FEATURES

- Prisma/Drizzle with TypeScript
- Native fetch for external APIs

11. Environment Configuration

DEFINITION

Use environment variables for ports, URLs and secrets; one config module.

SYNTAX

```
process.env.PORT
```

EXAMPLE

```
const port = process.env.PORT ?? 3000;
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Keeping keys in a locker, not on the shop counter; env variables keep config outside code.

தமிழ்

சாவிகளை கவுண்டரில் வைக்காமல் லாக்கரில் வைப்பது போல env variables config-ஐ குறியீட்டுக்கு வெளியே வைக்கின்றன.

PROS

- Safer secrets
- Per-environment config

CONS

- Missing vars crash at runtime

ADVANCED / NEW FEATURES

- Node `--env-file` flag
- Validated config with Zod

12. CORS and Cookies

DEFINITION

Allow browser apps from other origins with `cors`; use `cookie-parser` and secure cookie flags.

SYNTAX

```
app.use(cors({ origin: '...' }))
```

EXAMPLE

```
app.use(cors({ origin: 'https://myapp.com', credentials: true }));
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A guard who lets only invited guests from specific houses enter; CORS controls which websites can call your API.

தமிழ்

குறிப்பிட்ட வீடுகளிலிருந்து அழைக்கப்பட்ட விருந்தினர்களை மட்டும் அனுமதிக்கும் காவலர் போல CORS எந்த வலைத்தளங்கள் API-ஐ அழைக்கலாம் என்பதைக் கட்டுப்படுத்தும்.

PROS

- Browser security
- Fine-grained origins

CONS

- Misconfig breaks front-end calls

ADVANCED / NEW FEATURES

- `SameSite` cookie defaults
- Credentialed CORS rules

Advanced Level

13. Authentication and Authorization

DEFINITION

Register/login with hashed passwords, JWT or sessions, `authenticate` middleware and role checks.

SYNTAX

```
jwt.verify(token, secret)
```

EXAMPLE

```
function auth(req, res, next) {
  const t = req.headers.authorization?.split(' ')[1];
  try { req.user = jwt.verify(t, SECRET); next(); } catch { res.sendStatus(401); }
}
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A hostel gate pass: show your pass at each entry instead of explaining who you are every time.

தமிழ்

விடுதி வாயில் அனுமதிச் சீட்டு போல ஒவ்வொரு நுழைவிலும் சீட்டு காட்டினால் போதும்.

PROS

- Stateless
- Role checks easy

CONS

- Token theft risk
- Revocation is harder

ADVANCED / NEW FEATURES

- Passkeys
- Short-lived access + refresh tokens

14. File Uploads

DEFINITION

Handle multipart forms with Multer; validate type and size; store on S3 for production.

SYNTAX

```
upload.single('file')
```

EXAMPLE

```
const upload = multer({ dest: 'uploads/', limits: { fileSize: 2e6 } });
app.post('/avatar', upload.single('file'), handler);
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Parcels delivered to your shop are weighed and checked before accepting; Multer checks size and type of uploaded files.

தமிழ்

கடைக்கு வரும் பார்சல்கள் எடை, வகை சோதிக்கப்பட்டபின் ஏற்கப்படுவது போல Multer பதிவேற்றக் கோப்புகளின் அளவு, வகையைச் சோதிக்கிறது.

PROS

- Simple uploads
- Stream to storage

CONS

- Dangerous without limits
- Disk storage in containers is ephemeral

ADVANCED / NEW FEATURES

- Presigned S3 uploads
- Busboy streaming

15. Security Hardening

DEFINITION

Helmet, rate limiting, input sanitising, HTTPS, CSRF protection for cookies, secure headers.

SYNTAX

```
app.use(helmet())
```

EXAMPLE

```
app.use(helmet());
app.use(rateLimit({ windowMs: 60_000, limit: 100 }));
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Locking the doors, windows and cash box; helmet and rate limiting harden your API.

தமிழ்

கதவு, ஜன்னல், பணப்பெட்டியைப் பூட்டுவது போல helmet, rate limiting API-ஐ உறுதிப்படுத்துகின்றன.

PROS

- Prevents common attacks
- Few lines of code

CONS

- Not a complete security solution

ADVANCED / NEW FEATURES

- Helmet 8 defaults
- `express-rate-limit` v7

16. Logging and Monitoring

DEFINITION

Request logging with morgan/pino, correlation ids and health endpoints.

SYNTAX

```
app.use(morgan('combined'))
```

EXAMPLE

```
app.use(morgan('combined'));
app.get('/health', (req, res) => res.json({ ok: true }));
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A CCTV and a visitors' register at the shop; logs tell what happened and when.

தமிழ்

கடையில் CCTV மற்றும் பார்வையாளர் பதிவேடு போல logs என்ன, எப்போது நடந்தது என்று சொல்லும்.

PROS

- Faster debugging
- Audit trail

CONS

- Large log volume

ADVANCED / NEW FEATURES

- pino structured logging
- OpenTelemetry tracing

17. Testing APIs

DEFINITION

Test routes with Supertest and Vitest/Jest; use a test database or mocks.

SYNTAX

```
request(app).get('/x').expect(200)
```

EXAMPLE

```
const res = await request(app).get('/api/todos');
expect(res.status).toBe(200);
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A mock customer visits your shop before launch to check everything works.

தமிழ்

தொடக்கத்துக்கு முன் மாதிரி வாடிக்கையாளர் கடைக்கு வந்து எல்லாம் சரியா என்று பார்ப்பது போல.

PROS

- Safe refactors
- Documentation by tests

CONS

- Setup of test DB

ADVANCED / NEW FEATURES

- Supertest with Vitest
- Node test runner

18. API Documentation

DEFINITION

Document with OpenAPI/Swagger UI so frontend teams can try endpoints.

SYNTAX

```
app.use('/docs', swaggerUi.serve, swaggerUi.setup(spec))
```

EXAMPLE

```
app.use('/docs', swaggerUi.serve, swaggerUi.setup(spec));
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A menu card for developers listing every dish (endpoint) and how to order it.

தமிழ்

டெவலப்பர்களுக்கான மெனு அட்டை: ஒவ்வொரு endpoint-உம் எப்படி அழைப்பது என்பதும் பட்டியலிடப்பட்டிருக்கும்.

PROS

- Self-serve integration
- Try it in browser

CONS

- Docs drift if not generated

ADVANCED / NEW FEATURES

- OpenAPI 3.1
- Scalar/Redoc UIs

Master Level

19. Layered Architecture

DEFINITION

Routes -> controllers -> services -> repositories, with dependency injection and DTO validation.

SYNTAX

```
routes -> controllers -> services -> repositories
```

EXAMPLE

```
controllers/ services/ repositories/ middlewares/ routes/
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A hospital with reception, doctors, pharmacy and records; each layer has one role.

தமிழ்

மருத்துவமனையில் வரவேற்பு, மருத்துவர், மருந்தகம், பதிவுகள்; ஒவ்வொரு அடுக்குக்கும் ஒரு பங்கு.

PROS

- Testable
- Maintainable

CONS

- More boilerplate

ADVANCED / NEW FEATURES

- NestJS/Fastify provide opinionated structure

20. Caching, Queues and Performance

DEFINITION

Cache with Redis, compress responses, paginate, use background jobs, cluster mode.

SYNTAX

```
app.use(compression())
```

EXAMPLE

```
app.use(compression());
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Keeping popular items near the counter and packing parcels tightly; caching and compression make responses faster.

தமிழ்

பிரபலமான பொருட்களை கவுண்டர் அருகே வைத்து பார்சலை இறுக்கமாகக் கட்டுவது போல caching, compression பதில்களை வேகமாக்குகின்றன.

PROS

- Faster APIs
- Lower cost

CONS

- Cache invalidation is hard

ADVANCED / NEW FEATURES

- HTTP caching headers
- Redis
- Brotli compression

21. Deployment with Docker and CI/CD

DEFINITION

Containerise, use environment secrets, reverse proxy with Nginx, automated tests and deploys.

SYNTAX

```
docker build -t api .
```

EXAMPLE

```
docker build -t api . && docker run -p 3000:3000 api
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Sending the finished product in a standard container to any port; Docker and CI/CD make deployment repeatable.

தமிழ்

முடிந்த பொருளை தரப்படுத்தப்பட்ட கொள்கலனில் எந்தத் துறைமுகத்துக்கும் அனுப்புவது போல Docker, CI/CD deployment-ஐ மீண்டும் செய்யக்கூடியதாக்குகின்றன.

PROS

- Repeatable deploys
- Easy rollback

CONS

- Pipeline maintenance

ADVANCED / NEW FEATURES

- GitHub Actions with OIDC
- Container platforms (Fly, Render, Cloud Run)

22. Real Project: Task/Blog/E-commerce API

DEFINITION

Full REST API with auth, roles, validation, database, tests, docs and deployment.

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A full backend for a small business: users, orders, payments and docs.

தமிழ்

சிறு வணிகத்துக்கான முழு backend: பயனர்கள், ஆர்டர்கள், கட்டணம், ஆவணங்கள்.

PROS

- Good portfolio project

CONS

- Takes time

ADVANCED / NEW FEATURES

- Use Express 5 + TypeScript + Zod + Prisma

Interview Question Bank

Q1. What is Express?
A minimal, flexible Node.js web framework providing routing and middleware.

Q2. What is middleware?
A function with `(req, res, next)` that can modify the request/response or end the cycle.

Q3. How does error handling work?
Error-handling middleware has four arguments and must be registered after routes.

Q4. `app.use` vs `app.get`?
`use` mounts middleware for all methods/paths (prefix); `get` handles only GET for a path.

Q5. req.params vs req.query vs req.body?
Route segments, URL query string, and parsed request payload.

Q6. How do you secure an Express API?
Helmet, validation, auth (JWT), rate limiting, CORS configuration, HTTPS.

Q7. What is CORS?
A browser mechanism restricting cross-origin requests; the server opts in with headers.

Q8. How do you structure a large app?
Routers, controllers, services and repositories with config and middleware folders.

Q9. How do you handle async errors?
Use `async` handlers with a wrapper or Express 5, which forwards rejected promises to error middleware.

Q10. PUT vs PATCH?
PUT replaces a resource; PATCH changes part of it.

Capstone Project Ideas

1. Todo REST API
2. Blog API with auth
3. Library management API
4. E-commerce backend
5. URL shortener